这是一个非常经典的算法面试/竞赛题，通常被称为**“动态维护数据流的中位数”**。

1. 核心思想：把数据“劈成两半”

要找到中位数，我们不需要把所有数都排好序，我们只需要知道：

- **左边的一堆数（较小的一半）：** 它们的**最大值**是多少？
- **右边的一堆数（较大的一半）：** 它们的**最小值**是多少？

只要维护好这两堆数，中位数就在这两个堆的堆顶。

- **大根堆（Max-Heap）：** 存放较小的那一半数。堆顶就是左半部分的“最大值”。
- **小根堆（Min-Heap）：** 存放较大的那一半数。堆顶就是右半部分的“最小值”。

2. 维护规则（口诀）

为了让中位数永远在堆顶，我们要遵守两个简单的平衡原则：

1. **大小平衡：** 两个堆的大小差不能超过 1（即：`大根堆.size()` 要么等于 `小根堆.size()`，要么比它多 1）。
2. **数值有序：** 大根堆里所有的数都必须小于等于小根堆里的数。

3. 操作流程（代码模板）

每当来一个新的数 `x`：

1. **先判断放哪：** 如果 `x` 比大根堆的堆顶小，就扔进大根堆；否则扔进小根堆。
2. **再平衡：** 调整两个堆，保证它们的 `size` 差值保持在 0 或 1。
3. **查询：**
    - 如果两个堆总数是奇数，中位数就是大根堆的堆顶（因为它多存了一个）。
    - 如果两个堆总数是偶数，中位数就是（大根堆堆顶 + 小根堆堆顶）/ 2。

4. C++ 实战代码

```cpp
#include <queue>
#include <vector>
#include <iostream>

using namespace std;

priority_queue<int> max_heap; // 左半部分：大根堆
priority_queue<int, vector<int>, greater<int>> min_heap; // 右半部分：小根堆

void addNum(int num) {
    // 1. 放入逻辑
    if (max_heap.empty() || num <= max_heap.top()) {
        max_heap.push(num);
    } else {
        min_heap.push(num);
    }

    // 2. 平衡逻辑：保证 max_heap.size() >= min_heap.size() 且差值不超过 1
    if (max_heap.size() > min_heap.size() + 1) {
        min_heap.push(max_heap.top());
        max_heap.pop();
    } else if (min_heap.size() > max_heap.size()) {
        max_heap.push(min_heap.top());
        min_heap.pop();
    }
}

double findMedian() {
    if (max_heap.size() > min_heap.size()) return max_heap.top();
    return (max_heap.top() + min_heap.top()) / 2.0;
}
```

5. 为什么 A 组考这个？

- **时间复杂度优秀：** 插入一个数只需 $O(\log N)$，查询中位数只需 $O(1)$。即使有 10^6 个数，这套逻辑也跑得飞快。
- **空间复杂度低：** 只用了 $O(N)$ 的空间，且只存了当前数据。
- **思维训练：** 它考查了你对“动态维护”和“堆”结构的深刻理解。

学习建议：

1. **先搞定“对顶堆”逻辑：** 你可以找一道叫《动态中位数》或者 LeetCode 295 题练手，把上面的代码跑通。
2. **注意细节：** 平衡逻辑里的 `if` 判断顺序很关键，一定要保证 `max_heap` 的大小要么等于 `min_heap` ，要么比它大 1。
3. **A组重点：** 除了这种双堆，A 组还可能考“利用对顶堆处理滑动窗口问题”，逻辑是一样的，只是增加了“移除旧元素”的步骤（如果需要删除中间元素，记得用 `multiset` 代替堆）。

理解了这个，你在应对“动态最值”问题时就非常有底气了！这个模板在考场上是可以直接默写的。